Azentio Banking System

VaaniDesk Banking Assistant

Complete Backend API Specification & Frontend Integration Guide

Document Version: 1.0.0
Owner / Author: Member 1 — Senior Backend Engineer
Target Audience: Member 2 (AI Orchestration), Member 3 (React Frontend)
Environment Base URL: http://127.0.0.1:8000
API Canonical Prefix: /api/v1
OpenAPI Interactive Docs: http://127.0.0.1:8000/docs
Date of Issue: October 31, 2026
Classification: Internal Engineering Documentation

Notice: This document defines the canonical HTTP contract for the VaaniDesk AI-powered voice banking customer assistant. All voice/chat sessions, customer verification procedures, mock Loan Origination System (LOS), Loan Management System (LMS), and Core Banking System (CBS) APIs must conform to the input and output schemas detailed herein.

Table of Contents

• 1. Project Overview & System Architecture

• 2. Environment & Server Setup Instructions

• 3. Complete API Endpoint Reference (Detailed Input & Output)

• 3.1 System Health Probes

• 3.2 Customer Voice & Chat Session Management

• 3.3 Mock Banking Services (LOS, LMS, CBS)

• 3.4 Customer Service Management (CSM Tickets)

• 3.5 Operations Dashboard APIs

• 3.6 Preserved Customer Web Portal Endpoints

• 4. API Endpoint Summary Matrix

• 5. Frontend Developer Integration Guide

• 6. End-to-End User Journeys & Wireflows

• 7. Standardized Error Handling & Fault Envelope

• 8. Data Protection & Security Safeguards

• 9. Frontend Team Handoff Protocol

1. Project Overview & System Architecture

VaaniDesk is an enterprise-grade AI-powered voice and digital assistant for retail banking. It automates customer inquiries regarding loan statuses, monthly EMI repayment schedules, account balances, transaction disputes, and human escalations. Built with an asynchronous FastAPI backend and dual-driver SQLite/PostgreSQL architecture, it operates with zero external database dependencies in local development while maintaining seamless cloud readiness.

Architectural Separation:
• **Member 1 (Backend):** Core FastAPI engine, SQLite database schema, synthetic data seeding, server-controlled session state, customer identity verification, mock banking APIs, idempotent ticket creation, audit logging, and the AI adapter.
• **Member 2 (AI Orchestration):** Speech-to-Text, intent classification, multi-turn state machine, and Text-to-Speech generation via app.integrations.ai_pipeline.
• **Member 3 (Frontend):** React/Vite user interface for voice interactions, customer portals, and operations.

2. Environment & Server Setup Instructions

The backend is self-contained. The local SQLite database is created automatically upon first run.

```
# 1. Activate Virtual Environment
cd azentio/server
.venv\Scripts\activate          # Windows
```

```
source .venv/bin/activate       # Linux/macOS

# 2. Populate Repeatable Synthetic Banking Data
python -m app.db.seed

# 3. Run FastAPI Application
python run.py                   # Starts server on http://127.0.0.1:8000
# Or via uvicorn directly:
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

3. Complete API Endpoint Reference

3.1 System Health Probes

GET /health (Canonical Infrastructure Health Probe)

• **Authentication:** Public. No headers or payload required.
• **Success Response (200 OK):**

```
{
  "status": "ok",
  "service": "vaanidesk-backend"
}
```

3.2 Customer Voice & Chat Session Management

POST /api/v1/sessions (Initialize Assistant Session)

Creates a server-side session and generates a corresponding call audit record. The session starts in UNVERIFIED state.

INPUT (Optional Body):

```
{
  "customer_id": "C1023",
  "channel": "voice"
}
```

OUTPUT (201 Created):

```
{
  "session_id": "SES-3f92a10b",
  "call_id": "CALL-7e81b29c",
  "customer_id": "C1023",
  "current_state": "UNVERIFIED",
  "verified": false,
  "failed_verification_attempts": 0,
  "locked": false,
  "created_at": "2026-10-31T12:00:00Z"
}
```

POST /api/v1/sessions/{session_id}/verify (Customer Verification)

Verifies customer date of birth (DOB) against the synthetic banking database. Enforces server-side tracking of failed attempts. Exactly three (3) failed attempts lock the session permanently.

INPUT:

```
{
  "customer_id": "C1023",
  "dob": "1995-03-12"
```

```
}
```

OUTPUT (200 OK - Successful Verification):

```
{
  "session_id": "SES-3f92a10b",
  "verified": true,
  "customer_id": "C1023",
  "customer_name": "Alex Mercer",
  "current_state": "VERIFIED",
  "message": "Customer identity verified successfully."
}
```

OUTPUT (401 Unauthorized - Mismatch):

```
{
  "error": {
    "code": "AUTH_MISMATCH",
    "message": "Date of birth does not match customer record. 2 attempts remaining.",
    "retryable": true
  }
}
```

OUTPUT (403 Forbidden - Lockout Triggered):

```
{
  "error": {
    "code": "LOCKED",
    "message": "Session locked due to 3 failed verification attempts. Please contact human support.",
    "retryable": false
  }
}
```

POST /api/v1/sessions/{session_id}/turn (Process Conversation Turn)

Sends speech transcript or chat message to Member 2's AI orchestration adapter. Persists the user input, detected intents, assistant reply, and audit events.

INPUT:

```
{
  "text": "What is the status of my loan application?",
  "input_type": "voice_transcript"
}
```

OUTPUT (200 OK):

```
{
  "turn_id": "TURN-1029",
  "session_id": "SES-3f92a10b",
  "call_id": "CALL-7e81b29c",
  "turn_number": 1,
  "reply_text": "Your Personal Loan application APP-77 is currently in Document Verification.",
  "detected_intents": ["los_status_query"],
  "state": "VERIFIED",
  "verified": true,
  "tickets": [],
  "escalation": null
}
```

POST /api/v1/sessions/{session_id}/escalate (Escalate to Human Agent)

Transfers call to human support queue and generates an escalation ticket.

INPUT:

```
{
```

```
  "reason": "Customer requested human supervisor regarding disputed transaction",
  "category": "complaint",
  "priority": "HIGH"
}
```

OUTPUT (200 OK):

```
{
  "session_id": "SES-3f92a10b",
  "call_id": "CALL-7e81b29c",
  "escalated": true,
  "escalation_id": "ESC-5012",
  "ticket_id": "SR-9011",
  "message": "Call successfully transferred to human agent queue."
}
```

POST /api/v1/sessions/{session_id}/end (Close Session)

Closes session and updates call outcome to RESOLVED.

OUTPUT (200 OK):

```
{
  "session_id": "SES-3f92a10b",
  "call_id": "CALL-7e81b29c",
  "status": "ENDED",
  "message": "Session terminated successfully."
}
```

3.3 Mock Banking Services (LOS, LMS, CBS)

GET /api/v1/mock/los/applications?customer_id=C1023

Requires verified session header X-Session-ID. Returns loan applications and document checklist.

OUTPUT (200 OK):


```
{
  "customer_id": "C1023",
  "applications": [
    {
      "application_id": "APP-77",
      "product_type": "personal",
      "requested_amount": 300000.0,
      "application_status": "DOC_VERIFICATION",
      "pending_documents": ["salary_slip"],
      "created_at": "2026-10-15T09:30:00Z"
    }
  ],
  "source": "LOS-SIMULATED"
}
```


GET /api/v1/mock/lms/loans/{loan_id}

Returns loan balance, EMI amount, next due date, and overdue amount.

OUTPUT (200 OK):


```
{
  "loan_id": "LN-9012",
  "customer_id": "C1023",
  "application_id": "APP-70",
  "product_type": "home_loan",
  "outstanding_amount": 2450000.0,
  "emi_amount": 28500.0,
  "next_due_date": "2026-11-05",
  "overdue_amount": 0.0,
  "loan_status": "ACTIVE",
  "source": "LMS-SIMULATED"
}
```


GET /api/v1/mock/lms/loans/{loan_id}/schedule

Returns complete synthetic EMI schedule.

OUTPUT (200 OK):


```
{
  "loan_id": "LN-9012",
  "schedule": [
    {
      "schedule_id": "SCH-101",
      "installment_number": 1,
      "due_date": "2026-10-05",
      "amount": 28500.0,
      "payment_status": "PAID"
    },
    {
      "schedule_id": "SCH-102",
      "installment_number": 2,
      "due_date": "2026-11-05",
      "amount": 28500.0,
      "payment_status": "PENDING"
    }
```

```
  ],
  "source": "LMS-SIMULATED"
}
```

GET /api/v1/mock/cbs/accounts/{account_id}

Returns savings/checking account balance with masked account number.

OUTPUT (200 OK):

```
{
  "account_id": "ACC-1001",
  "customer_id": "C1023",
  "masked_account": "****8821",
  "account_type": "savings",
  "balance": 145000.5,
  "currency": "INR",
  "account_status": "ACTIVE",
  "source": "CBS-SIMULATED"
}
```

3.4 Customer Service Management (CSM Tickets)

POST /api/v1/mock/csm/tickets (Idempotent Ticket Creation)

Enforces database uniqueness on idempotency_key. Retrying with identical key returns the existing ticket without creating duplicates.

INPUT:

```
{
  "customer_id": "C1023",
  "type": "service_request",
  "category": "repayment_schedule",
  "summary": "Customer requested repayment schedule PDF via email",
  "priority": "NORMAL",
  "call_id": "CALL-7e81b29c",
  "idempotency_key": "SES-3f92a10b-turn2"
}
```

OUTPUT (201 Created or 200 OK on repeat):

```
{
  "ticket_id": "SR-4471",
  "status": "OPEN",
  "customer_id": "C1023",
  "type": "service_request",
  "category": "repayment_schedule",
  "summary": "Customer requested repayment schedule PDF via email",
  "priority": "NORMAL",
  "idempotency_key": "SES-3f92a10b-turn2",
  "created_at": "2026-10-31T12:05:00Z"
}
```

3.5 Operations Dashboard APIs

- **GET /api/v1/ops/calls**: List calls with pagination (limit, offset).
- **GET /api/v1/ops/calls/{call_id}**: Complete call audit with conversation turns and trace.
- **GET /api/v1/ops/tickets**: List tickets filtered by status, priority, category.
- **GET /api/v1/ops/escalations**: List open escalations requiring agent intervention.

4. API Endpoint Summary Matrix

Method	Endpoint	Purpose	Auth	Status
GET	/health	Canary health check	None	Tested
GET	/api/v1/health	System health & timestamp	None	Tested
POST	/api/v1/sessions	Create voice session	None	Tested
POST	/api/v1/sessions/{id}/verify	Verify DOB / Lockout	Session	Tested
POST	/api/v1/sessions/{id}/turn	Process speech turn	Session	Tested
POST	/api/v1/sessions/{id}/escalate	Human agent handoff	Session	Tested
POST	/api/v1/sessions/{id}/end	End voice call	Session	Tested
GET	/api/v1/mock/los/applications	LOS applications	Verified	Tested
GET	/api/v1/mock/lms/loans/{id}	LMS loan balance	Verified	Tested
GET	/api/v1/mock/lms/loans/{id}/schedule	LMS EMI schedule	Verified	Tested
GET	/api/v1/mock/cbs/accounts/{id}	CBS account balance	Verified	Tested
GET	/api/v1/mock/cbs/transactions/{id}	CBS transaction info	Verified	Tested
POST	/api/v1/mock/csm/tickets	Create ticket (idempotent)	Verified	Tested
GET	/api/v1/mock/csm/tickets/{id}	Get ticket details	Verified	Tested
GET	/api/v1/ops/calls	Ops call history	Staff	Tested
GET	/api/v1/ops/calls/{id}	Call trace & turns	Staff	Tested
GET	/api/v1/ops/tickets	Ops tickets list	Staff	Tested
GET	/api/v1/ops/escalations	Ops active escalations	Staff	Tested

5. Frontend Developer Integration Guide

To integrate with React/Vite, make HTTP requests to http://127.0.0.1:8000/api/v1. Maintain the session_id in React state for the entire voice or chat call.

```
// Sample React Hook for VaaniDesk Assistant Call
import { useState } from 'react';

export function useVaaniSession() {
  const [session, setSession] = useState(null);

  const startSession = async () => {
    const res = await fetch('http://127.0.0.1:8000/api/v1/sessions', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ channel: 'web_voice' })
    });
    const data = await res.json();
    setSession(data);
    return data;
  };

  const verify = async (customerId, dob) => {
```

```
    const res = await fetch(`http://127.0.0.1:8000/api/v1/sessions/${session.session_id}/verify`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ customer_id: customerId, dob })
    });
    const data = await res.json();
    if (res.ok) setSession(prev => ({ ...prev, verified: true, customer_id: customerId }));
    return { ok: res.ok, data };
  };

  return { session, startSession, verify };
}
```

6. Standardized Error Handling & Fault Envelope

All error responses adhere to a consistent JSON envelope with actionable error codes:

```
{
  "error": {
    "code": "AUTH_MISMATCH",
    "message": "Date of birth does not match customer record.",
    "retryable": true
  }
}
```

7. Data Protection & Security Safeguards

1. **Zero Real Customer Data:** All customer, loan, and financial records are strictly synthetic.
2. **Server Verification Authority:** The backend never trusts client-supplied verified: true flags.
3. **PII Redaction:** Full account numbers are automatically masked (e.g. ****8821) and credentials/secrets are excluded from audit records.